Mixed-Precision Training & Loss Scaling, Worked Out by Hand

Why fp16 needs a scaled loss, and what it saves

A complete numerical walk-through of *why* half-precision training breaks tiny gradients, and *how* loss scaling repairs the damage with one multiplication and one division. We dissect the bit layouts of `float32` and `float16`, locate the exact value where small gradients fall out of fp16's representable range, and then push a real gradient through the scale → backward → unscale → step pipeline. Every number below was produced by the NumPy reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It takes one stubborn gradient, $g = 3 \times 10^{-7}$, and follows it through fp16 storage. It shows that g has slipped below fp16's smallest *normal* value 6.104×10^{-5}, so it survives only as a lossy subnormal — and that a slightly smaller gradient 1×10^{-8} vanishes to exactly 0. Then it applies a loss scale $S = 2^{16} = 65536$, watches $S\,g = 0.01966$ land comfortably inside fp16's normal range, and divides back by S to recover g for the `float32` master-weight update.

How to read this. Each idea gets a *Mini-example* box isolating the operation on the cleanest possible numbers, alongside the running gradient. Read this to understand what `autocast` + `GradScaler` are really doing underneath the two-line API the course shows; the memory accounting at the end is the same bytes = numel × itemsize rule from the Tensors topic, applied to a model.

Concepts covered, in order: the fp32 and fp16 bit layouts · dynamic range vs precision · the smallest representable fp16 numbers · why small gradients underflow · loss scaling as a multiply/-divide pair · recovering the true gradient for the fp32 master copy · memory: 4 MB vs 2 MB per million activations · overflow to `inf` and the dynamic scaler · what must stay in fp32.

1 The setup: two number formats, one gradient

Mixed precision runs the heavy linear algebra of the forward and backward passes in `float16` (fast, half the bytes) while keeping a `float32` master copy of the weights for the optimizer to update. The whole scheme lives or dies on one fact: a `float16` number cannot represent very small magnitudes. So we first need the exact anatomy of both formats.

Property	`float32` (fp32)	`float16` (fp16)
Total bits	32	16
Sign / exponent / mantissa	1/8/23	1/5/10
Bytes per element	4	2
Largest finite value	3.403×10^{38}	65504
Smallest *normal* value	1.175×10^{-38}	6.104×10^{-5}
Smallest *subnormal* value	$\sim 1.4 \times 10^{-45}$	5.960×10^{-8}
Decimal digits of precision	~ 7	~ 3

The two columns differ in *two* independent ways. The 5 exponent bits give fp16 a much narrower **dynamic range** (it tops out at 65504 and bottoms out near 6×10^{-5} for normal numbers). The 10 mantissa bits give it much coarser **precision** (only $\sim$3 decimal digits). Both will bite us below.

The running example. During a backward pass a particular weight receives the gradient

$$g \;=\; 3 \times 10^{-7}.$$

This is a perfectly ordinary small gradient deep in a network. We will track exactly what fp16 does to it.

Intuition

The recipe of mixed precision is three moves. **(1)** Do the heavy matrix math in fast low precision (`float16`) to save time and memory. **(2)** Keep one high-precision `float32` *master copy* of the weights so that millions of tiny updates still accumulate correctly. **(3)** *Scale the loss* up before the backward pass so that the tiny gradients it produces are pushed up into the range fp16 can actually store — then scale them back down before they touch the master weights. The rest of this document is just those three moves, with numbers.

2 Step 1 — Where fp16 runs out of room

A normalized floating-point number is $\pm 1.m \times 2^e$. For `float16` the exponent field is 5 bits, which after the bias encodes unbiased exponents from -14 up to $+15$. The smallest *normal* magnitude is therefore

$$\boxed{2^{-14} \;=\; 6.103515625 \times 10^{-5} \;\approx\; 6.104 \times 10^{-5}}$$

Below 2^{-14} the format switches to *subnormals*: the leading bit is no longer an implied 1, and the values sit on a fixed grid with spacing

$$2^{-24} \;=\; 5.960 \times 10^{-8}.$$

The very smallest nonzero fp16 magnitude is one grid step, $2^{-24} = 5.960 \times 10^{-8}$. Anything closer to zero than half a step rounds to exactly 0.

Our gradient against this ruler. Compare $g = 3 \times 10^{-7}$ to the boundaries:

$$\underbrace{5.960 \times 10^{-8}}_{\text{smallest subnormal}} < \underbrace{3 \times 10^{-7}}_{g} < \underbrace{6.104 \times 10^{-5}}_{\text{smallest normal } 2^{-14}}.$$

So g has fallen *below the smallest normal value*. It is not zero — yet — but it can only be held as a coarse subnormal. Snapping g to the subnormal grid:

$$\frac{3 \times 10^{-7}}{2^{-24}} = 5.033\ldots \longrightarrow 5 \quad \Rightarrow \quad \text{fp16}(g) = 5 \times 2^{-24} = 2.980 \times 10^{-7}.$$

We have lost precision (only one significant digit survives), and we are one arithmetic step away from disaster: a gradient just a little smaller disappears entirely.

Mini-example — this operation in isolation

Take $g' = 1 \times 10^{-8}$, smaller than *half* a subnormal step ($\frac{1}{2} \cdot 2^{-24} = 2.980 \times 10^{-8}$). Rounding to the nearest grid point gives 0:

$$\text{fp16}(1 \times 10^{-8}) = 0.0.$$

This is **underflow**: a real, nonzero gradient becomes a hard zero, so the weight it belongs to receives *no update at all*. Multiply a few such factors together in a deep backward pass and whole regions of the network silently stop learning.

Watch out

The danger zone is everything below $2^{-14} = 6.104 \times 10^{-5}$. Plenty of legitimate gradients live there, especially late in training or in deep stacks where the chain rule multiplies many sub-one factors. In fp16 they are either crushed to a couple of significant bits (subnormal) or rounded to 0 (underflow). fp32, whose smallest normal is 1.175×10^{-38}, never notices these values — which is exactly why the *master* weights stay in fp32.

3 Step 2 — Loss scaling: lift the gradients into range

The fix is almost insultingly simple. Gradients are linear in the loss: if you multiply the loss $\mathcal{L}$ by a constant S before calling `backward`, every gradient comes out multiplied by the same S, because

$$\frac{\partial(S\,\mathcal{L})}{\partial w} = S\,\frac{\partial \mathcal{L}}{\partial w}.$$

Choose S large enough to shove the tiny gradients up past 2^{-14}, and they land where fp16 stores them faithfully. The standard starting value is a power of two so the multiply is exact:

$$\boxed{S = 2^{16} = 65536}$$

Our gradient, scaled. Apply S to g:

$$S\,g = 65536 \times 3 \times 10^{-7} = 0.0196608 \approx 0.01966.$$

Now check the ruler again: 0.01966 sits in the interval $[2^{-6}, 2^{-5})$, a fully *normal* fp16 region where the spacing is only

$$2^{-6-10} = 2^{-16} = 1.526 \times 10^{-5}.$$

Storing it:

$$\text{fp16}(0.0196608) \;=\; 0.0196533203125 \;\approx\; 0.01965.$$

That is full fp16 precision — about 3 good digits — instead of the single-digit subnormal mush we got before. The gradient is safe.

Intuition

Picture fp16's representable numbers as a row of fence posts that are dense near 1 and become sparse, then nonexistent, as you approach 0. A tiny gradient is a person standing in the gap before the first post — invisible to the format. Loss scaling does not move the fence; it *walks the person to the right* by a factor S until they stand among the dense posts, records their position there, and then walks the recorded position back left by the same S. Same gradient, but now it survived the round trip.

4 Step 3 — Unscale, then update the fp32 master weights

The scaled gradient $S\,g$ is the wrong size for an optimizer step — it is S times too big. Before `optimizer.step()` we must **unscale**: divide every gradient by the same S. This happens in `float32`, on the master copy, so no precision is lost on the way back:

$$\hat{g} \;=\; \frac{\text{fp16}(S\,g)}{S} \;=\; \frac{0.0196533203125}{65536} \;=\; 2.99886 \times 10^{-7}.$$

Compare to the true gradient $g = 3 \times 10^{-7}$:

$$\frac{|\hat{g} - g|}{g} \;=\; 0.038\%.$$

We recovered the gradient to within four parts in ten thousand — excellent for a value that, *unscaled*, fp16 could barely hold. The optimizer now applies $\hat{g}$ to the fp32 master weight, which is then cast down to fp16 for the next fast forward pass.

Mini-example — this operation in isolation

The underflow case rescued. Recall $g' = 1 \times 10^{-8}$, which fp16 crushed to 0. Scale it: $S\,g' = 65536 \times 10^{-8} = 0.00065536$, which is normal in fp16 and stores as $0.000655174\ldots$. Unscale: $0.000655174/65536 \;=\; 9.997 \times 10^{-9}$ — a gradient that was previously *lost entirely* is now recovered to within 0.03%. The weight that would have frozen now learns.

Stage	without scaling	with $S = 65536$
true gradient g	3×10^{-7}	3×10^{-7}
value sent to fp16	3×10^{-7}	$S\,g = 0.0196608$
stored as fp16	2.980×10^{-7} (subnormal)	0.0196533 (normal)
recovered for step $\hat{g}$	2.980×10^{-7}	2.99886×10^{-7}
relative error	lossy ($\sim$ 1 digit)	0.038%

5 Step 4 — The payoff: half the bytes

Why endure all this? Because fp16 halves memory, and memory is the binding constraint on GPU training. The accounting is the familiar rule

$$\text{bytes} \;=\; \text{numel} \times \text{itemsize}.$$

Take a single activation tensor of one million elements — a modest feature map:

$$\begin{aligned} \text{fp32:}\ & 1{,}000{,}000 \times 4 = 4{,}000{,}000 \text{ bytes} = 4.0 \text{ MB}, \\ \text{fp16:}\ & 1{,}000{,}000 \times 2 = 2{,}000{,}000 \text{ bytes} = 2.0 \text{ MB}. \end{aligned}$$

Exactly 2 MB saved on one tensor — a clean 50% cut. Scaled to a model's weights, a 100-million-parameter network costs 400 MB of fp32 weights versus 200 MB in fp16. Activations, which dominate VRAM during training, halve in the same way. On tensor-core GPUs the fp16 matmuls also run roughly 1.5–2× faster, so the win is memory *and* speed at once.

Tensor	numel	itemsize	total	relative
activation (fp32)	1,000,000	4 B	4.0 MB	1.0×
activation (fp16)	1,000,000	2 B	2.0 MB	0.5×
weights, 100M (fp32)	100,000,000	4 B	400 MB	1.0×
weights, 100M (fp16)	100,000,000	2 B	200 MB	0.5×

When this is the right choice

Modern GPU training with tensor cores is the home turf of mixed precision. On an NVIDIA A100/H100 (or any tensor-core card), `autocast` runs the big matmuls and convolutions in fp16/bf16 at full tensor-core throughput, roughly halving activation memory and giving a large speedup — often 1.5–2× end to end — with no change in final accuracy when paired with a `GradScaler` and an fp32 master copy. For large models it is not an optimization, it is what makes the batch fit at all.

6 Step 5 — Picking S automatically: the dynamic scaler

A fixed S is fragile. Too small and tiny gradients still underflow; too large and the *big* gradients overflow past fp16's ceiling of 65504 and become `inf`. **Dynamic loss scaling** adapts S on the fly with two rules:

- **Overflow ⇒ back off.** If any gradient is `inf` or `nan` after the backward pass, *skip the optimizer step entirely* (the gradients are garbage) and *halve* S.
- **Long stable run ⇒ push higher.** After a fixed number of consecutive non-overflowing steps, *double* S to keep the tiny gradients as far from underflow as possible.

A short trajectory (growth after every 3 stable steps, for illustration; the real default interval is 2000):

step	event	S after step	action
0	ok	65536	apply step
1	ok	65536	apply step
2	ok	131072	apply step, grow S ($\times 2$)
3	overflow	65536	**skip step**, halve S
4	ok	65536	apply step
5	ok	65536	apply step

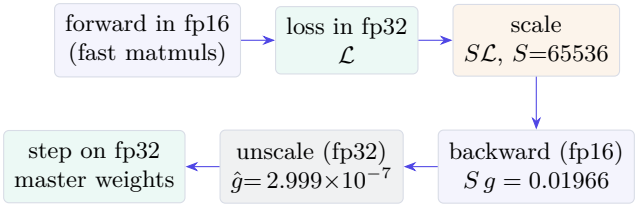
Notice that step 3's overflow is simply discarded — the weights do not move that iteration — and S drops from 131072 back to 65536. The scaler hunts for the largest S that does not overflow, riding just below the fp16 ceiling.

Watch out

fp16 **overflow** is the mirror image of underflow. The largest finite fp16 value is 65504; anything above it becomes `inf`. For example fp16(70000) = `inf`, and 65504×2 in fp16 is `inf`. Once a single gradient is `inf` or `nan`, the whole update is poisoned — which is precisely why the dynamic scaler *skips* that step rather than applying it. Always keep an fp32 master copy, and keep numerically sensitive ops — reductions, `softmax`, layer norms, and the loss itself — in fp32, where the range is enormous.

7 The full pipeline, drawn

One training iteration under mixed precision, with our numbers on the wire:



Read left-to-right, top row then bottom row. The loss is computed in fp32, multiplied by $S = 65536$, and only *then* differentiated; the scaled gradient $S\,g = 0.01966$ is large enough to store in fp16; it is divided by S back in fp32 to recover $\hat{g} = 2.999 \times 10^{-7}$; and that drives the step on the fp32 master weights, which are recast to fp16 for the next forward pass.

8 Mixed precision, at a glance

#	Quantity	Value here	Why it matters
1	fp16 smallest normal	$2^{-14} = 6.104\times10^{-5}$	below this, gradients degrade
2	fp16 smallest subnormal	$2^{-24} = 5.960\times10^{-8}$	below half this, $\to 0$
3	fp16 max finite	65504	above this, $\to$ `inf`
4	raw gradient g	3×10^{-7}	subnormal in fp16 (lossy)
5	loss scale S	$2^{16} = 65536$	power of two, exact multiply
6	scaled $S\,g$	0.0196608	now normal in fp16
7	stored fp16$(S\,g)$	0.0196533	~ 3 good digits
8	recovered $\hat{g}$	2.99886×10^{-7}	error 0.038%
9	memory, 1M elems	4 MB $\to$ 2 MB	50% cut

9 Equation cheat-sheet

fp16 smallest normal	$2^{-14} = 6.104 \times 10^{-5}$
fp16 subnormal step	$2^{-24} = 5.960 \times 10^{-8}$
fp16 max finite	65504
Gradient is linear in loss	$\partial(S\mathcal{L})/\partial w = S\,(\partial\mathcal{L}/\partial w)$
Scaled gradient	$g_{\text{scaled}} = S\,g$
Unscaled (for the step)	$\hat{g} = \text{fp16}(S\,g)\,/\,S$
Memory footprint	bytes = numel $\times$ itemsize
fp16 vs fp32 bytes	2 vs 4 per element (0.5$\times$)
Dynamic scaler, overflow	skip step, $S \leftarrow S/2$
Dynamic scaler, stable run	$S \leftarrow 2S$

Appendix A Reference implementation

Every number above is reproducible from the following program. The first block is the exact arithmetic (NumPy, float64 for the truth values, float16 to model fp16 storage); the second is the PyTorch `autocast` + `GradScaler` sketch the course shows, which automates the scale / unscale / skip logic.

```
import numpy as np

# --- fp16 boundaries ---
print(np.finfo(np.float16).max)    # 65504.0   (largest finite)
print(2.0**-14)                    # 6.103515625e-05  (smallest NORMAL)
print(2.0**-24)                    # 5.96e-08         (subnormal step)

# --- the raw gradient underflows / degrades in fp16 ---
g = 3e-7
print(float(np.float16(g)))        # 2.980e-07  -> lossy subnormal (1 digit)
print(float(np.float16(1e-8)))     # 0.0        -> underflow to zero

# --- loss scaling: multiply the loss (hence every grad) by S ---
S = 2**16                          # 65536
sg = S * g                         # 0.0196608
print(sg)                          # 0.0196608  -> now NORMAL in fp16
sg16 = float(np.float16(sg))
print(sg16)                        # 0.0196533203125  (~3 good digits)

# --- unscale in float32 to recover the gradient for the step ---
g_hat = sg16 / S
print(g_hat)                       # 2.99886e-07
print(abs(g_hat - g) / g * 100)    # 0.038  (percent error)

# --- memory: bytes = numel * itemsize ---
numel = 1_000_000
print(numel * 4 / 1e6)             # 4.0 MB  (float32)
print(numel * 2 / 1e6)             # 2.0 MB  (float16)

# --- overflow: above 65504 -> inf ---
print(float(np.float16(70000)))    # inf


# ====================================================================
# PyTorch automatic mixed precision (AMP): autocast + GradScaler.
# GradScaler does scale / unscale / skip-on-overflow / adjust-S for you.
# ====================================================================
import torch
from torch.amp import autocast, GradScaler

scaler = GradScaler('cuda', enabled=torch.cuda.is_available())

for X_b, y_b in loader:
    optimizer.zero_grad()
    with autocast(device_type='cuda',
                  enabled=torch.cuda.is_available()):
        output = model(X_b)            # fp16 matmuls (fast, half memory)
        loss   = criterion(output, y_b)  # loss kept in fp32 internally

    scaler.scale(loss).backward()      # multiply loss by S, then backward
    scaler.unscale_(optimizer)         # divide grads by S (fp32 master)
```

```
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)            # skips the step if grads are inf/nan
    scaler.update()                   # grow S on stable runs, halve on overflow
```

— end of document —